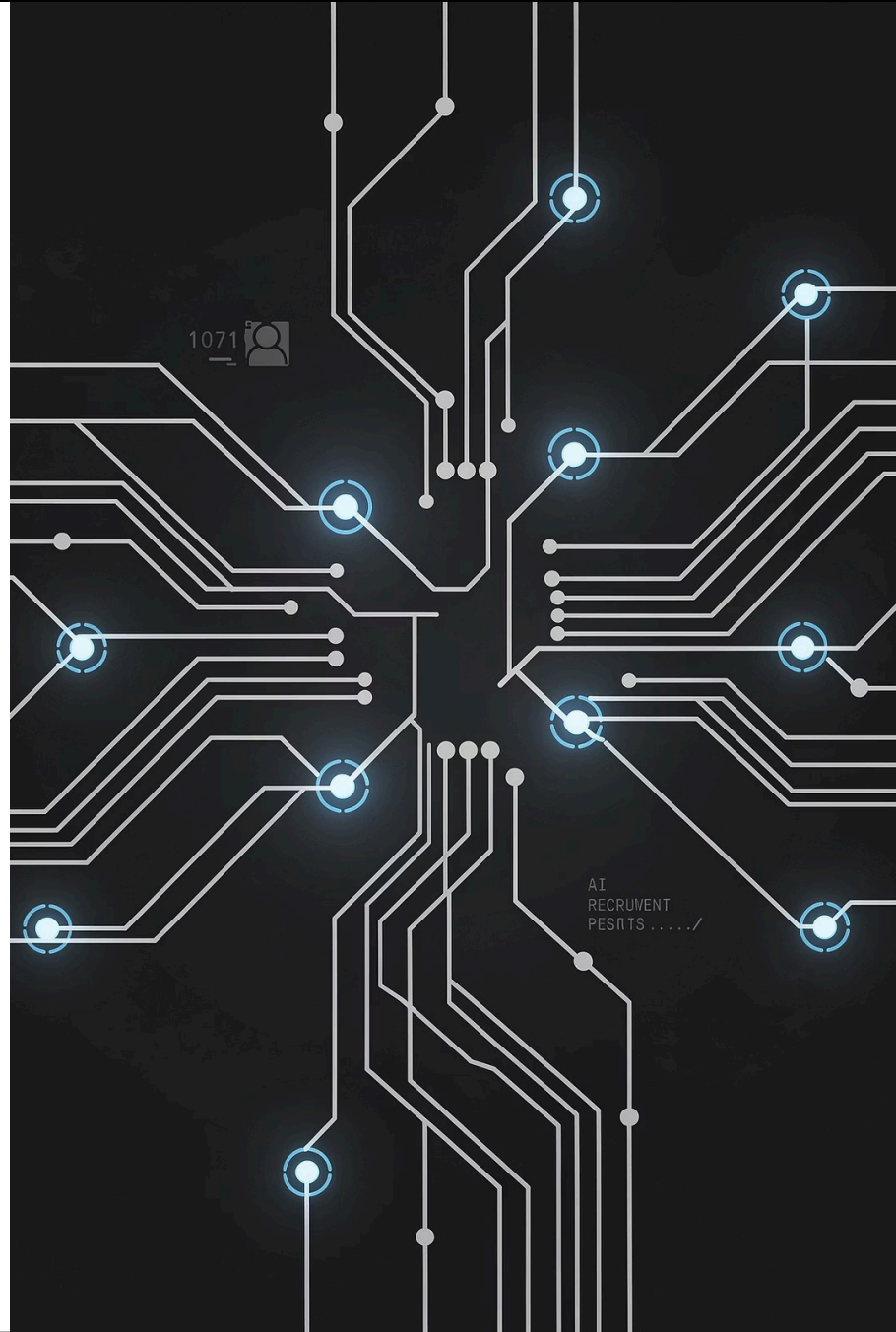
AI-Powered Candidate Matching System

Improving speed, consistency, and quality in resume screening — a smarter way to find the right talent.

Github Link

https://github.com/healthycarrot/prodapt-capstone



The Core Problem

Finding consistent, quality candidates efficiently remains recruitment's greatest challenge.

⏱️ Speed

Manual screening creates significant bottlenecks with large applicant volumes.

⚖️ Consistency

Varied evaluation criteria cause unpredictable hiring outcomes.

🎯 Quality

Keyword matching often fails to identify relevant skills accurately.

We must improve hiring speed, consistency, and quality simultaneously.

Design Thinking

The goal isn't simply to score candidates — it's to give recruiters a UX that makes it easy to understand why a candidate is the right fit.

Semantic Understanding

Requirements are interpreted semantically, moving beyond simple keyword parsing.

Transferable Skill Recognition

Candidates match via related and transferable capabilities, not exact overlap.

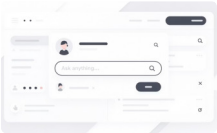
Explainability First

Natural language explanations support, rather than replace, human recruiter judgement.

Format-Agnostic Profiles

Diverse resume formats are normalized for consistent, fair comparison.

Key Features

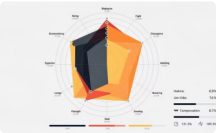


Natural Language Search

Enter hiring requirements as free text. The system semantically retrieves the most relevant candidates.

Hard Filters

Apply structured constraints — location, experience, education — to refine the candidate pool with precision.



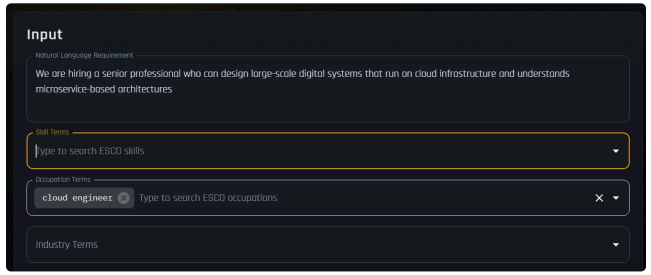
Multi-Dimensional Scoring

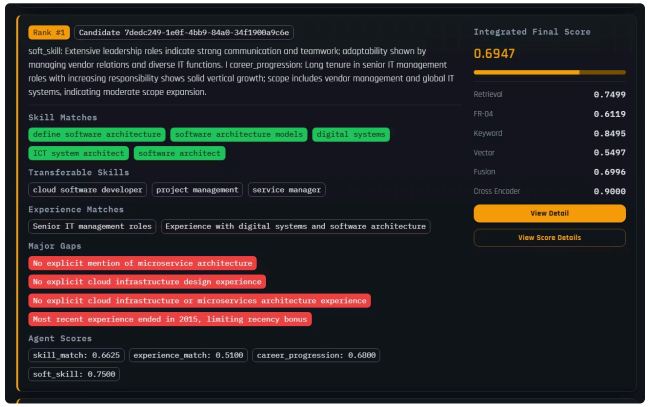
Candidates are scored across skills, experience, career progression, and soft skills — not just a single rank.

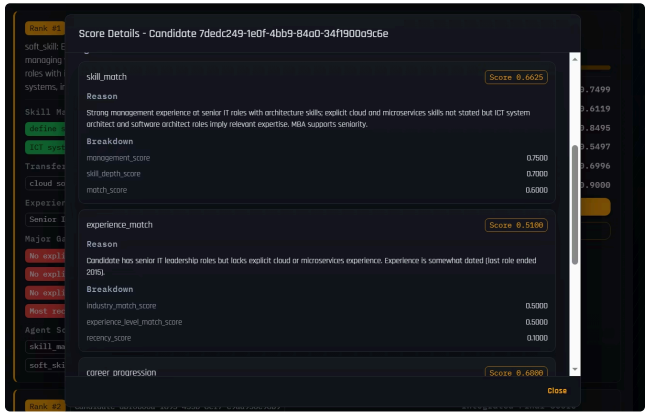
Full Candidate Access

Review natural language score explanations alongside summarised profiles and original source resumes.

Product Demo

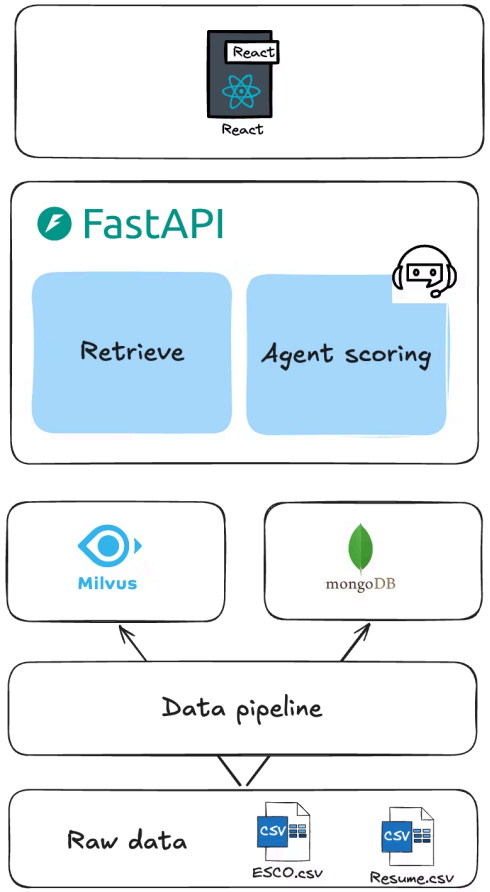






ARCHITECTURE & DESIGN

Enterprise Architecture



Design Intentions

Microservice Thinking

Each capability is independently deployable and replaceable.

— **Separable Retrieval:** The retrieve capability is intentionally decoupled from the frontend search flow.

— **Low Switching Cost:** The agent layer and vector database can be swapped without rebuilding the full system.

Modularity and future extensibility are first-class design goals.

ARCHITECTURE & DESIGN

System Thinking: Data Challenges

The Nature of This Use Case

1 Multiple format Sources

Resume data arrives from multiple sources in different formats — HTML, PDF, plain text, structured JSON.

2 Unstructured by Default

The majority of candidate data is unstructured, making consistent extraction non-trivial.

3 Naive Chunking Fails

Simple token-length chunking degrades retrieval quality. Passing whole documents inflates context size, increasing latency and cost.

Therefore, the **core design focus is the preprocessing pipeline** — before retrieval or scoring can be effective.

PIPELINE DESIGN

Pipeline Stage 1: Structuring

Raw resumes in varying formats are parsed and decomposed into structured, retrievable elements.

HTML Parsing

HTML resumes are parsed using semantic tags to extract sections cleanly and reliably.

NLP / LLM Parsing

Non-HTML formats use NLP-based title extraction or LLM-based parsing to identify section boundaries.

Key Field Extraction

Skill and Occupation fields are extracted as structured elements, enabling downstream RAG and filtering.

PIPELINE DESIGN

Pipeline Stage 2: Normalisation

> **"By normalising to the ESCO dataset, we can handle data from diverse formats with high accuracy in RAG retrieval."**

The Challenge

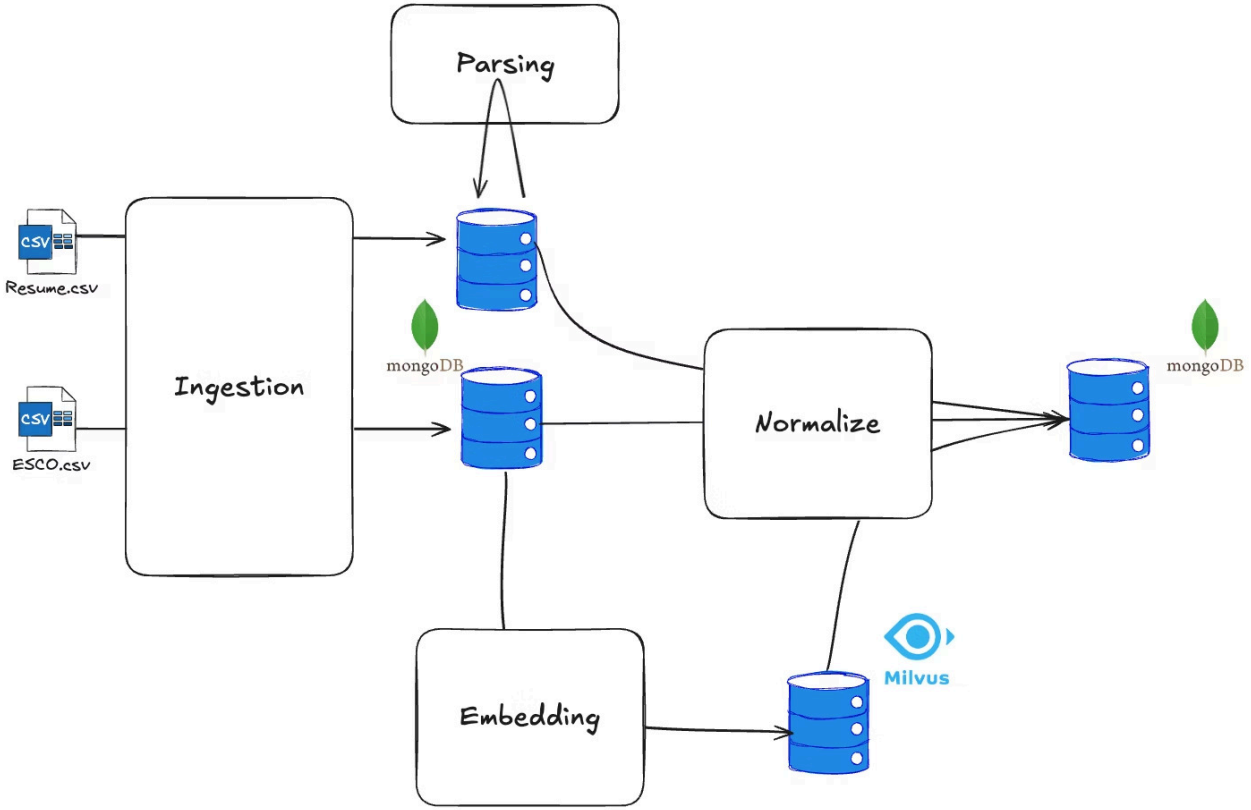
Datasets often use different terminology for identical skills and job titles.

The Approach

We normalise all candidate data to the ESCO taxonomy for consistent, standardised classification.

Why ESCO?

Use the **ESCO dataset** as the shared taxonomy for skills, competencies, qualifications, and occupations



AI WORKFLOW DESIGN

Retrieval Design

A multi-stage pipeline combining filtering, hybrid search, and cross-encoder reranking.

1

Hard Filter Extraction

Filters explicit selections and normalises input to ESCO standard.

2

Hybrid Search

Runs parallel vector and keyword searches for broad, precise recall.

3

Fusion Score

Merges search paths into a single unified ranked result list.

4

Cross-Encoder Reranking

Reranks top candidates for maximum relevance before final scoring.

AI WORKFLOW DESIGN

Agent-Based Scoring

Five Specialist Agents

- Skill Match
- Experience Match
- Education Match
- Career Progression
- Soft Skills

Skill Match Agent

Compares query requirements vs. candidate skills. Outputs: score, breakdown, and rationale.

Dimensions

Skill Match · Skill Depth · Management Relevance

Default Weights

50 / 25 / 25

Notes

- Weights adapt to query priorities
- Missing skills can be inferred from the role
- Transferable skills add positive value

Business Value & ROI

⚡ Operational Efficiency

Reduces time on manual screening, freeing recruiters for higher-value decisions.

🎯 Hiring Quality

Surfaces strong candidates beyond keyword overlap, including transferable skills.

⚖️ Consistency

Reduces evaluator variance and standardises reasoning behind every recommendation.

Measuring Success: KPIs

Operational Metrics

Average Review Time

Time saved per application review.

Hiring Lead Time

Time from job posting to offer acceptance.

Rediscovery Rate

High-quality candidates found from past pools.

Quality Metrics

Interview Pass-Through Rate

AI-matched candidates advancing through stages.

Top-Rank Conversion Rate

Success rate of top-ranked candidates.

Inter-Rater Agreement

Recruiter consistency using AI insights.

Track impact on hiring efficiency and quality.

FUTURE ROADMAP

Future Extensibility

CI/CD Setup

Establish automated pipelines for continuous integration and deployment.

Batch Eval

Scale evaluation runs through batch processing for efficiency.

Agent Scoring Tuning

Adjust agent scoring weights based on evaluation results.

Multi-Source Validation

Validate system performance across multiple data sources.

GitHub URL

https://github.com/healthycarrot/prodapt-capstone

ARCHITECTURE & DESIGN

Trade-off Decisions: Database Selection

Milvus: Comparison with Alternatives

Category	⭐ **Milvus**	Pinecone	Weaviate	Qdrant	pgvector
Model	✅ **OSS + Managed**	Fully Managed	OSS + Cloud	OSS + Cloud	PG Extension
Scalability	✅ **High (distributed)**	High	High (complex)	High (lightweight)	Limited by PG
Overhead	✅ **High**	Low	Medium	Low–Med	Low
Filter Search	✅ **Strong (scalar)**	Strong (metadata)	Strong	Strong (payload)	SQL-based

MongoDB as system of record + Milvus as retrieval engine = extensibility and performance.